ITS Symposium manuscript No.
(will be inserted by the editor)

ReRAPTOR: Reverse-Objective RAPTOR for Time-Budgeted Value-Maximizing Transit Itinerary Planning

Bin Zhou · Guangxian Li · Teruki Ogo · Yutaka Arakawa · Tsunenori Mine

Received: date / Accepted: date

Abstract Conventional public-transit routing prioritizes arrival time, whereas a leisure itinerary may instead need to maximize collected value under a hard time budget. We formulate this timetable-constrained problem and propose ReRAPTOR, a heuristic search in the Round-bAsed Public Transit Optimized Router (RAPTOR) family that preserves schedule feasibility while ranking partial itineraries by a pluggable value model. Candidate trimming and bucket beam jointly form a two-stage Beam layer; heuristic Dominance removes locally weaker labels, and an optional penalty down-ranks undesirable route patterns. On Fukuoka timetable data, a seeded stop-reward benchmark compares ReRAPTOR with nine routing and metaheuristic baselines. ReRAPTOR has the highest mean Stop Score (872.438), 8.710% above the non-dominated sorting genetic algorithm II (NSGA-II), while NSGA-II requires 6.89× as much generation time on average. This aggregate gain is associated mainly with reaching more distinct scored stops and returning feasible routes more reliably; among tasks on which every NSGA-II run succeeds, the score difference is inconclusive. Ablation identifies the Beam layer and Dominance as the main scalability controls. Sensitivity results are unchanged for $W \geq 25$ and all tested label-ranking coefficients, whereas the candidate cap governs the quality–resource trade-off. These results establish a strong overall balance on the tested network, not universal superiority.

Keywords Transit itinerary planning · Transit schedule data · RAPTOR · Time-budgeted route planning · Multi-objective search

Bin Zhou and Guangxian Li
Graduate School of Information Science and Electrical Engineering, Kyushu University, Fukuoka, Japan
E-mail: zhou.bin.883@s.kyushu-u.ac.jp; li.guangxian.021@s.kyushu-u.ac.jp

Teruki Ogo
Faculty of Information Science and Engineering, Kyushu Sangyo University, Fukuoka, Japan
E-mail: k25rm021@st.kyusan-u.ac.jp

Yutaka Arakawa and Tsunenori Mine
Faculty of Information Science and Electrical Engineering, Kyushu University, Fukuoka, Japan
E-mail: arakawa@ait.kyushu-u.ac.jp; mine@ait.kyushu-u.ac.jp

1 Introduction

Public-transit routing usually seeks earliest-arrival or Pareto-efficient journeys [2,4,6,12]. For leisure travel, however, a user may want the most rewarding itinerary that can be completed within a fixed time. Its value may depend on the stops already visited, their rewards, and the order in which they are reached. This task is therefore closer to budgeted itinerary design than to shortest-path search [11,21,22].

Combining a timetable with a history-dependent objective creates a specific search problem. Every extension must respect scheduled departures, arrivals, and transfers, yet partial itineraries that reach the same stop can have different collected value and future opportunities. Keeping only the earliest arrival can discard a valuable history; keeping every history causes rapid label growth. Existing timetable routers and value-maximizing trip planners address the two sides of this problem, but their usual state and search controls do not directly resolve this tension.

We address the gap with ReRAPTOR, a reverse-objective RAPTOR-style search over timetable schedules [6,24,25]. Here *reverse-objective* does not mean reversing time or graph direction. It means retaining timetable feasibility as a hard constraint while making

route value, rather than earliest arrival, the primary ranking objective. ReRAPTOR carries the history required by a pluggable value module and controls the resulting state growth through a two-stage Beam layer (candidate trimming and bucket beam), heuristic Dominance, and optional penalty-based down-ranking.

We ask whether ReRAPTOR improves the quality–time balance and where it loses, which controls produce its behavior, and whether fixed hyperparameters drive the conclusion. A reproducible Fukuoka benchmark with nine baselines, a full ablation, 22 sensitivity configurations, and three route cases answers these questions. The contributions are a formal value-maximizing timetable problem, an explicit pluggable search algorithm, and evidence that separates aggregate performance, mechanism, robustness, and limits. ReRAPTOR is heuristic: it searches a controlled subset of feasible itineraries and has no global optimality guarantee.

2 Related Work

Round-Based Public Transit Optimized Router (RAPTOR) is a round-based framework for earliest-arrival and multicriteria transit routing [6]. Later work improves transfers, restricted Pareto search, and acceleration [1, 2, 4, 5, 12, 19]. Fare-aware and broader multi-objective variants have also been studied [8, 23], supported by the General Transit Feed Specification (GTFS) and related timetable datasets [10, 14, 17, 25]. These methods make timetable routing efficient, but do not directly optimize history-dependent route value under a fixed total budget.

Tourist trip design and orienteering instead maximize collected value under time constraints [9, 11, 21, 22]. Preference-aware route systems extend this idea to personalized and multimodal settings [18, 20, 27, 26]. General optimizers such as the nondominated sorting genetic algorithm II (NSGA-II), iterated local search (ILS), variable neighborhood search (VNS), and simulated annealing (SA) are also relevant [3, 13, 15, 16]. ReRAPTOR differs by optimizing value during timetable-feasible expansion, not only after full itineraries have been constructed.

3 Problem Formulation

Let $G = (\mathcal{S}, \mathcal{E})$ be a timetable graph, where $\mathcal{S}$ is the stop set and $\mathcal{E}$ is the set of scheduled ride edges. An edge $e = (u, v, \tau^{\mathrm{dep}}, \tau^{\mathrm{arr}}, p, \ell)$ has origin $u(e) = u$, destination $v(e) = v$, departure and arrival times, trip p, and line ℓ. A query $q = (o, t_0, B)$ gives the origin, earliest departure, and time budget. A route $r = (e_1, \ldots, e_m)$, where $m \geq 1$ is its number of scheduled ride legs, is timetable-feasible when the two inter-leg conditions below hold for $i = 1, \ldots, m-1$:

$$\begin{aligned} u(e_1) = o, \qquad \tau^{\mathrm{dep}}(e_1) \geq t_0, \\ v(e_i) = u(e_{i+1}), \\ \tau^{\mathrm{dep}}(e_{i+1}) \geq \tau^{\mathrm{arr}}(e_i) + \delta(e_i, e_{i+1}), \\ \tau^{\mathrm{arr}}(e_m) \leq t_0 + B. \end{aligned} \tag{1}$$

Here $\delta(e_i, e_{i+1}) = 0$ if both rides remain on the same trip and line, and equals the minimum transfer time Δ_{tr} otherwise. An objective module identified by θ supplies a complete-route value $F_\theta(r)$, a resource function T_θ for partial and complete routes, a partial-label ranking score S_θ, objective-state updates, and a Boolean acceptance rule $A_\theta(r)$. Let $\mathcal{R}_\theta(q)$ contain the routes that satisfy Eq. 1, $T_\theta(r) \leq B$, and $A_\theta(r) = 1$. The target problem is

$$r^* \in \arg\max_{r \in \mathcal{R}_\theta(q)} F_\theta(r). \tag{2}$$

This interface separates the task definition from the common timetable search.

Experiments 1–3 use a seeded stop-reward module. Each stop receives a fixed reward $w_\omega(s)$. Let $V(r)$ be the set of stops in r, $V^+(r) = V(r) \setminus \{o\}$, and route elapsed time $T(r) = \tau^{\mathrm{arr}}(e_m) - t_0$, which includes initial waiting. Every unique non-origin stop also incurs a per-stop effective-time charge η. This is a resource-accounting charge, not physical dwell inserted into the timetable. The effective time and complete-route value are

$$\begin{aligned} T_\eta(r) &= T(r) + \eta |V^+(r)|, \qquad T_\eta(r) \leq B, \\ F_{\omega,\eta}(r) &= \sum_{s \in V^+(r)} w_\omega(s). \end{aligned} \tag{3}$$

The seed ω fixes one reward map. Replacing the reward with an attraction rating or other utility changes the module, not timetable feasibility. $A_{\omega,\eta}$ accepts a resource-feasible route with at least two legs. The per-stop charge is added only to the resource budget and does not shift the scheduled connection times in Eq. 1.

4 Proposed Method

4.1 Labels and Timetable-Feasible Expansion

ReRAPTOR uses RAPTOR-style round buckets but expands scheduled ride segments directly [6, 12]. Here a *round* is one leg-extension depth, not the transfer-count round of the original RAPTOR algorithm. A label

$$\lambda = (v, a, k, \pi, \mathbf{c}, \rho, z_\theta, s_\theta) \tag{4}$$

stores current stop v, arrival time a, leg count k, a predecessor chain π, stop-visit counts $\mathbf{c}$, accumulated in-vehicle minutes ρ, objective state z_θ, and its cached ranking score $s_\theta = S_\theta(\lambda)$. The predecessor chain determines the ordered route $r(\lambda)$ and its last trip. For the stop-reward module, $z_{\omega,\eta} = (V^+, \sigma)$, where $\sigma = \sum_{s\in V^+} w_\omega(s)$. Bucket $\mathcal{L}_k(v)$ contains labels at stop v after k legs; $\mathcal{M}_k$ is the set of stops whose round-k buckets still require expansion.

The initial label λ_0 has $v = o$, $a = t_0$, $k = 0$, $\pi = \varnothing$, $\rho = 0$, the module's zero objective state, and $s_\theta = S_\theta(\lambda_0)$. For the stop-reward module, $V^+ = \varnothing$, $\sigma = 0$, $c_o = 1$, and $c_s = 0$ for every $s \in \mathcal{S} \setminus \{o\}$.

An extension appends a scheduled edge only when its origin is v, its departure satisfies Eq. 1, and the updated objective-specific resource remains feasible. In the stop-reward module, the partial effective time is $T_\eta(\lambda) = a - t_0 + \eta|V^+|$. A first visit adds its reward and the per-stop effective-time charge; a repeated visit adds neither. This state retains information that an earliest-arrival label alone cannot represent.

4.2 Value Ranking and Search Control

The objective module ranks every feasible partial label by

$$S_\theta(\lambda) = \Phi_\theta(\lambda) - P(\lambda), \tag{5}$$

where Φ_θ is the objective-specific surrogate before optional route-pattern penalties. The term P only lowers rank; it does not make a label infeasible. For the stop-reward task,

$$\begin{aligned} S_{\omega,\eta}(\lambda) = {} & 10000\sigma + 5\,\mathrm{SPH} - 0.1T_\eta(\lambda) \\ & - 50000[T_\eta(\lambda) - B]_+ - P(\lambda), \end{aligned} \tag{6}$$

where $[x]_+ = \max(0, x)$. Score per hour $\mathrm{SPH} = 60\sigma/T_\eta(\lambda)$ when $T_\eta(\lambda) > 0$, and is zero for the initial label. The hard feasibility check normally makes the budget term zero; it remains a safeguard in the shared scorer. The route-pattern penalty is

$$\begin{aligned} P(\lambda) = {} & 50000\mathbb{I}\left[\max_{s\in\mathcal{S}} c_s > 2\right] \\ & + 20000\mathbb{I}\left[|V^+| > 0 \ \wedge \ \rho < 2|V^+|\right], \end{aligned} \tag{7}$$

where c_s is the visit count of stop s, and $\mathbb{I}[\cdot]$ is one when its condition holds and zero otherwise. The second condition is written without division so that it is defined for the initial label, where $|V^+| = 0$. We call this optional term *Negative down-ranking*. The reward term sets the main ordering, while SPH and effective time resolve nearties. These fixed coefficients are tested in Experiment 3.

The Beam layer limits growth in two stages. Candidate trimming gives same-trip continuation first priority, then orders edges by the following objective score and earlier schedule times. For edge e, let $d_e = \tau^{\mathrm{arr}}(e) - \tau^{\mathrm{dep}}(e) + \eta$, let $w_e = 0$ when $v(e) = o$ and $w_\omega(v(e))$ otherwise, and let $\alpha_e = 0.15$ when $v(e)$ has already been visited and 1 otherwise:

$$\begin{aligned} H(e \mid \lambda) = {} & \alpha_e w_e + 25\frac{w_e}{\max(1, d_e)} - 0.05\max(1, d_e) \\ & + 2\mathbb{I}[\text{same trip}]. \end{aligned} \tag{8}$$

It keeps $C(B) = 30$ candidates for $B \le 240$ min and 20 otherwise. *Heuristic Dominance* then compares labels in the same stop–round bucket. Write $\sigma(\lambda)$ for the accumulated reward stored in the objective state of λ. Then

$$\begin{aligned} \lambda^a \succeq \lambda^b \iff {} & \sigma(\lambda^a) \ge \sigma(\lambda^b) \ \wedge \ T_\eta(\lambda^a) \le T_\eta(\lambda^b) \\ & \wedge \big(\sigma(\lambda^a) > \sigma(\lambda^b) \ \vee \ T_\eta(\lambda^a) < T_\eta(\lambda^b) \\ & \vee \ S_{\omega,\eta}(\lambda^a) > S_{\omega,\eta}(\lambda^b)\big). \end{aligned} \tag{9}$$

Thus, an exact reward–time tie is resolved only by a strictly higher $S_{\omega,\eta}$. This rule is not Pareto-safe: it may discard a partial label that could lead to a Pareto-optimal completed route because visited sets and continuation states can differ. It is a deliberate heuristic reduction [7,12]. The Beam layer's second stage, bucket beam, keeps the best $W = 50$ labels by S_θ. Thus, `NoBeam` disables both candidate trimming and bucket beam. The default width is a preselected middle setting, not a fitted optimum.

In Algorithm 1, $v(e)$ is the edge destination; FeasibleEdges applies timetable, lookahead, and budget tests; TopCandidates applies H and $C(B)$; Extend updates Eq. 4; and InsertND applies Dominance, or always inserts when it is disabled. TopW truncates its bucket in place to the W largest-S_θ labels. Eligible is true exactly when $A_\theta(r(\lambda)) = 1$ and $[\lambda^{\mathrm{best}} = \varnothing \ \vee \ S_\theta(\lambda) > S_\theta(\lambda^{\mathrm{best}})]$. The largest-$S_\theta$ accepted label admitted by InsertND approximates Eq. 2; bucket trimming prevents later expansion of removed labels but does not erase an accepted solution already observed.

4.3 Complexity and Implementation

With both Beam stages enabled, let $C_k(v, \lambda) \le C(B)$ be the number of candidate edges retained for label $\lambda \in \mathcal{L}_k(v)$. The number of attempted extensions is bounded by

$$N_{\mathrm{ext}} \le \sum_{k=0}^{R-1} \sum_{v\in\mathcal{M}_k} \sum_{\lambda\in\mathcal{L}_k(v)} C_k(v, \lambda) \le R|\mathcal{S}|WC(B). \tag{10}$$

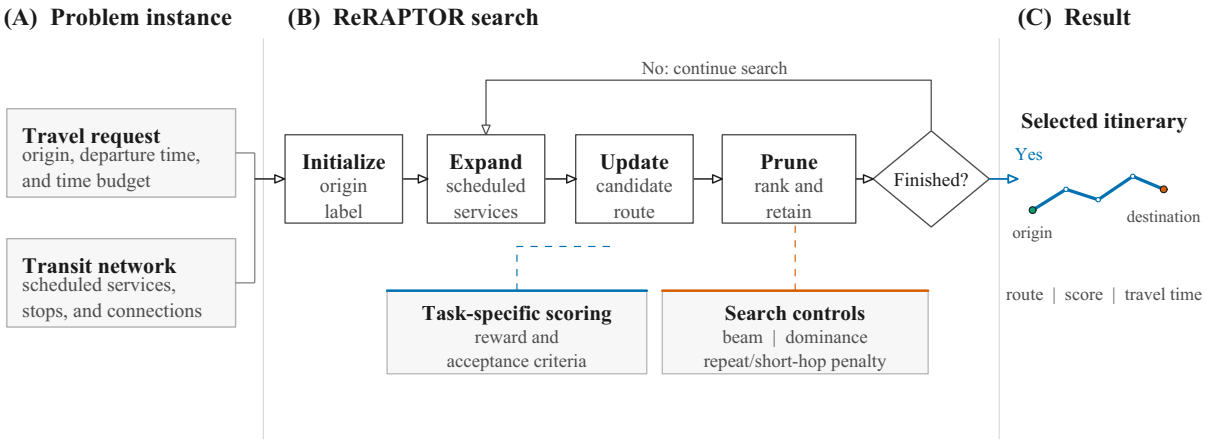


Fig. 1 ReRAPTOR workflow. A common timetable-feasible search backbone uses a task-specific scoring and acceptance module; Experiment 2 disables the Beam layer, Dominance, and Negative down-ranking individually and in combination.

Algorithm 1 ReRAPTOR controlled value search

Require: $q, \theta, W, C(B)$, and maximum depth R
initialize $\mathcal{L}_0(o) = \{\lambda_0\}$, $\mathcal{M}_0 = \{o\}$, and $\lambda^{\text{best}} = \varnothing$
for $k = 0, \ldots, R-1$ **do**
  $\mathcal{M}_{k+1} \leftarrow \varnothing$
  for all $v \in \mathcal{M}_k$ and $\lambda \in \mathcal{L}_k(v)$ **do**
    $E \leftarrow \textsc{FeasibleEdges}(\lambda, q)$
    if candidate trimming is enabled **then**
      $E \leftarrow \textsc{TopCandidates}(E, C(B), H)$
    end if
    for all $e \in E$ **do**
      $\lambda' \leftarrow \textsc{Extend}(\lambda, e, \theta)$
      if $T_\theta(\lambda') \le B$ **then**
        compute $S_\theta(\lambda')$
        if $\textsc{InsertND}(\lambda', \mathcal{L}_{k+1}(v(e)))$ **then**
          if bucket beam is enabled **then**
            $\textsc{TopW}(\mathcal{L}_{k+1}(v(e)), W, S_\theta)$
          end if
          $\mathcal{M}_{k+1} \leftarrow \mathcal{M}_{k+1} \cup \{v(e)\}$
          if $\textsc{Eligible}(\lambda', \lambda^{\text{best}}, \theta)$ **then**
            $\lambda^{\text{best}} \leftarrow \lambda'$
          end if
        end if
      end if
    end for
  end for
  if $\mathcal{M}_{k+1} = \varnothing$ **then**
    break
  end if
end for
return λ^{best}, or failure if none was accepted

Dominance checks at most W labels per insertion, giving a conservative $O(R|\mathcal{S}|C(B)W^2)$ bound, excluding timetable lookup and objective updates. Active buckets contain $O(|\mathcal{S}|W)$ labels; retaining all rounds gives $O(R|\mathcal{S}|W)$ label records.

The reference implementation indexes outgoing schedules, uses a 120-min schedule-query lookahead window (distinct from the Greedy baseline's L), sets $\Delta_{\text{tr}} = 3$ min, and limits depth to $R = \min(220, \max(40, \lfloor B/2 \rfloor + 20))$. Resource guards terminate a run with its current best accepted label; their incidence is reported rather than hidden.

5 Experimental Evaluation

5.1 Common Data, Measures, and Protocol

The experiments follow the questions stated in the Introduction. Experiment 1 evaluates quality, runtime, reliability, and conditions of underperformance; Experiment 2 isolates the search controls; Experiment 3 tests hyperparameters; and Experiment 4 explains representative route outcomes.

Baselines. Baseline selection follows three criteria: (1) task compatibility with timetable-feasible fixed-budget routing, (2) reproducible implementation in our evaluation pipeline, and (3) diversity across optimizer families. We therefore compare general-purpose metaheuristics (NSGA-II [3], ILS [15], VNS [16], and SA [13]), transit-routing baselines (multicriteria RAPTOR (McRAPTOR), Pure RAPTOR [6], and Greedy), and two recent adapted methods described below. ILS uses a timetable-feasible 2-opt-like cut-and-tail-regrowth neighborhood. Pure RAPTOR is the same scheduled-edge expansion with Beam, Dominance, and Negative down-ranking disabled but a 60-s guard retained. Greedy uses $L = 3$, the number of one-step onward candidates included in its discounted lookahead. Supplementary Table S7 lists the fixed implementation

parameters for all nine baselines; the source files named there are the executable specifications.

Adapted baselines. Adaptive large-neighborhood search (ALNS) and dynamic preference (DynPref) address related systems. Table 1 states what was retained and changed. All methods use the same timetable, query grid and execution order, reward maps, effective-time and transfer rules, route validator, and zero-score failure policy. Methods that modify complete routes also use the same timetable-feasible repair interface. Because iteration, stagnation, and resource guards remain method-specific, search effort is not exactly equalized; ALNS and DynPref are adaptations, not original-setting replications.

Tasks and reporting. The frozen input is the Fukuoka timetable snapshot for the Wednesday service date 24 December 2025 (`segments_fukuoka_20251224.csv`). It contains 304,344 timetable segments, 1,032 stops, and 302 lines. The grid combines 10 origins, three departures (07:00, 10:00, 14:00), four budgets (2h, 4h, 6h, 8h), and five reward seeds. An origin–departure–budget triple is a *base query* (120 total); adding a reward seed gives a *matched task* (600 per method). After sorting stop codes, each seed $\omega \in \{20260323, \ldots, 20260327\}$ draws $w_\omega(s) \sim \mathcal{U}\{1, \ldots, 100\}$ independently once per stop, where $\mathcal{U}$ denotes the discrete-uniform distribution on these integers. The map is then fixed across methods. A unique non-origin stop is scored once and adds a per-stop effective-time charge $\eta = 10$ min; the origin adds neither reward nor this charge. Route elapsed time $T(r)$ includes initial waiting. The primary outcome is $F_{\omega,\eta}(r)$, called *Stop Score*. Score per hour is $\mathrm{SPH} = 60F_{\omega,\eta}(r)/T_\eta(r)$.

Stochastic methods use algorithm seeds $0, \ldots, 4$, averaged within each matched task; deterministic methods run once. We then average reward seeds within each base query. Invalid or empty routes remain zero-score failures, so the primary comparison measures both route quality and the ability to return a feasible result. All methods share inputs and validation but retain their own search guards. Runs were sequential on Windows 11 (build 26200), Intel Core i5-12600KF (10 physical/16 logical cores), 31.84 GiB (gibibytes) memory, Python 3.12.7, NumPy 1.26.4, pandas 2.2.2, SciPy 1.13.1, and NetworkX 3.3. Wall time excludes planner/data loading and post-return route validation: it is the performance-counter interval around the route-generation solver call. No query-level parallelism was used. Experiments were separate batches, so wall times are compared only within tables. Paired 95% intervals use 20,000 crossed block-bootstrap replications: reward-seed levels and origin–departure blocks are resampled independently with replacement, and each replicate averages their crossed task weights.

5.2 Experiment 1: Quality, Runtime, and Reliability

5.2.1 Overall Results and Score Interpretation

Table 2 gives the overall (2h+4h+6h+8h) query-level means. ReRAPTOR has the highest mean Stop Score (872.438), 8.710% above NSGA-II (802.539); NSGA-II requires 6.89× as much generation time (35634.781 versus 5171.181 ms). These comparisons are directly calculable from Table 2: $872.438 - 802.539 = 69.899$, $69.899/802.539 \times 100 = 8.710\%$, and $35634.781/5171.181 = 6.89$.

The absolute difference of 69.899 points is 8.710% of the NSGA-II mean. Because the benchmark draws each stop reward from $\mathcal{U}\{1, \ldots, 100\}$, with expectation 50.5, dividing the gap by 50.5 gives a reward-equivalent of 1.384 average-valued benchmark stops ($69.899/50.5 = 1.384$; 95% crossed block-bootstrap confidence interval (CI) $[0.159, 2.592]$; Supplementary Table S1). It normalizes score units; it is not an observed stop count, monetary value, or measure of user satisfaction.

Observed task metrics support a coverage interpretation. ReRAPTOR visits 1.32 more unique scored stops on average (95% CI $[0.535, 2.092]$; Supplementary Table S1), an advantage under this positive-reward benchmark. However, its selected reward per scored stop is 1.914 points lower on the 581 tasks where NSGA-II returns at least one feasible route (95% CI $[-3.247, -0.654]$; Supplementary Table S1). Thus, the aggregate gain is associated with reaching more distinct stops and returning feasible routes reliably, not higher mean reward per stop.

ReRAPTOR beats NSGA-II in 380/600 matched tasks, ties in 49, and loses in 171; after reward-seed averaging, it has both higher score and lower time in 92/120 base queries (Fig. 2 and Supplementary Table S1).

Budget and seed robustness. ReRAPTOR is higher-scoring and faster than NSGA-II at every budget. Their mean scores are 398.61/327.98, 803.75/656.50, 988.91/965.94, and 1298.49/1259.73 for 2h–8h. The gap is also positive for all reward seeds (+41.55 to +117.62), and all 600 ReRAPTOR tasks return a positive feasible route (Supplementary Table S2, Panels A–C).

Reliability and conditions of underperformance. The mean paired gap is +69.899 points (95% crossed block-bootstrap CI $[+8.054, +130.917]$), but the 2h–8h gaps are +70.633, +147.243, +22.967, and +38.753; only

Table 1 Implementation of the two adapted baselines ($u = T_\eta/B$: fraction of the effective-time budget used). Complete parameter files accompany the code.

Method	Retained idea	Fixed-timetable adaptation	Search budget
ALNS (adapted)	Adaptive destroy–repair, operator weights, and SA acceptance [28].	Four tail cuts (shallow, medium, deep, restart) are repaired through the shared timetable interface for at most 44 steps. Exploration rates are 0.20/0.32/0.48/0.55; weights update every 12 iterations (reaction 0.30, rewards 5/2/0.5); initial SA acceptance temperature $T_0 = 6$, cooling 0.985, floor 0.6.	160 iterations or 45 s; seeds $0, \ldots, 4$.
DynPref (adapted)	Rolling shift from exploration to efficiency [27].	Shared-mobility state is removed. Early ($u < .35$), middle ($.35 \le u < .70$), and late stages use progressively shallower tail cuts, exploration rates 0.50/0.38/0.22, and a score that shifts from distinct-stop coverage to efficiency while reward remains primary.	24 failed repairs or 45 s; seeds $0, \ldots, 4$.

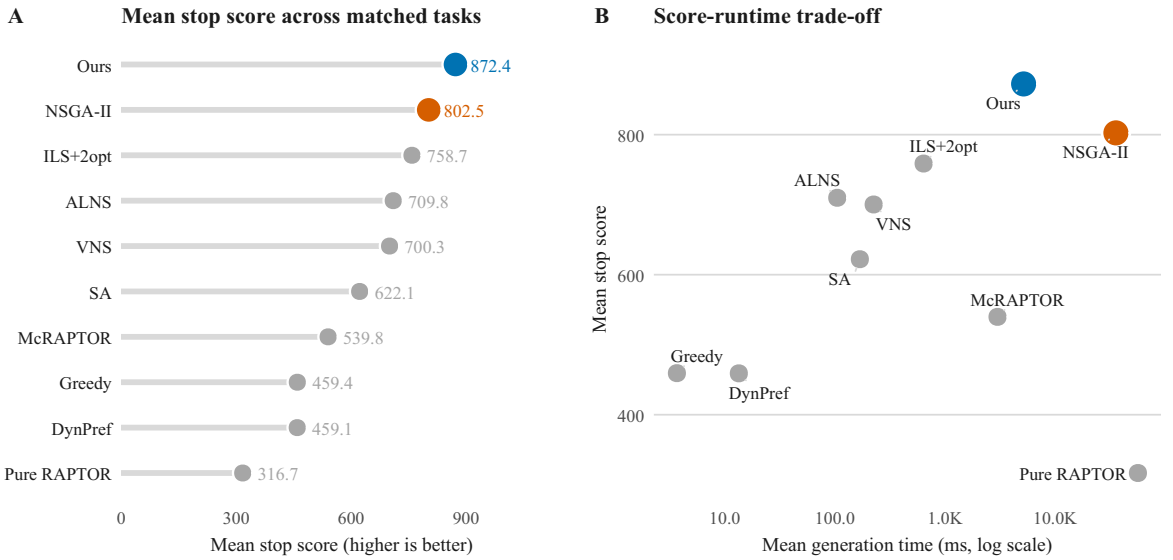


Fig. 2 Overview of Experiment 1 under the score-guided multi-seed benchmark. Panel A ranks methods by mean stop score, and Panel B shows the quality–generation-time landscape on a logarithmic time axis.

Table 2 Experiment 1 summary (120 query-level means). Generation is solver wall time; Rank averages within-query ranks (score descending, then time).

Method	Stop Score	Generation (ms)	Score / hour	Rank
ReRAPTOR	872.438	5171.181	185.473	1.775
NSGA-II	802.539	35634.781	164.945	3.200
ILS+2-opt	758.733	638.398	162.296	2.867
ALNS (adapted)	709.834	104.866	153.519	4.442
VNS	700.271	224.575	151.637	4.883
SA	622.115	168.314	134.631	6.475
McRAPTOR	539.763	2994.828	146.180	6.850
Greedy (L=3)	459.357	3.667	103.515	8.400
DynPref (adapted)	459.099	13.393	107.426	8.067
Pure RAPTOR	316.733	56470.874	163.958	8.042

the first two intervals exclude zero. The conditional mean loss is 236.423, and the worst task decile averages −556.350 (Supplementary Tables S1–S2). The descriptive subgroup means vary more across origins than across departure blocks; these exploratory subgroup intervals are not adjusted for multiple comparisons. Noke averages −225.693 (95% CI $[-361.123, -88.403]$) with 33/60 losses; its 6h and 8h gaps are −468.920 and −687.200. Ohori-koen and Yakuin also turn negative at 6h and 8h, although their origin intervals cross zero. Only the 10:00 departure block is clearly positive (+120.373, 95% CI $[14.141, 207.916]$); the subgroup values are tabulated in Supplementary Table S3. Figure 3 localizes the largest losses to long-budget origin cells but does not establish a topological cause.

NSGA-II has 288/3000 algorithm-seed failures affecting 148 tasks. On the 452 tasks where all five runs succeed, the mean gap is +4.708 (95% CI $[-63.024, 67.875]$); averaging its feasible seeds gives −3.059 over 581 tasks (95% CI $[-71.301, 62.440]$; Supplementary Table S1). Thus, the primary result includes feasible-route reliability; successful-run solution quality is inconclusive.

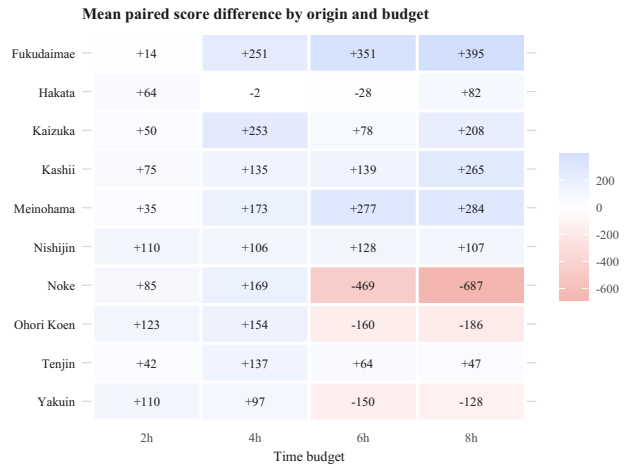


Fig. 3 Mean paired Stop Score gap (ReRAPTOR minus NSGA-II) by origin and budget. Each cell averages 15 matched tasks (three departures and five reward seeds); blue favors ReRAPTOR and red favors NSGA-II. Algorithm failures remain zero-score outcomes under the primary all-task policy.

Table 3 Canonical Experiment 2 ablation (means over 600 matched tasks). Generation is solver wall time; Labels counts the initial label plus every label created by edge extension before later Dominance/Beam removal; CapHit is the percentage stopped by the 60-s timeout or the triple-disabled configuration's 1,000,000-label cap.

Method	Stop Score	Generation (ms)	Labels	CapHit (%)
Ours (Full)	872.438	10288.03	91,447	4.67
Ours (NoNeg)	872.472	11433.06	93,642	5.50
Ours (NoDom)	787.458	24141.25	219,676	16.00
Ours (NoBeam)	705.717	43041.76	539,463	67.67
Ours (NoBeam-NoDom)	299.873	56597.52	882,480	91.67
Ours (NoDom-NoNeg)	798.042	20688.47	231,871	11.67
Ours (NoBeam-NoNeg)	790.943	42042.27	1,000,717	65.50
Ours (NoBeam-NoDom-NoNeg)	300.588	41234.67	903,851	91.17

5.3 Experiment 2: Three-Component Search-Control Ablation and Supporting Evidence

Experiment 2 disables the Beam layer, Dominance, and Negative down-ranking individually and jointly from `Full`, which has all three controls enabled, on the same 600 tasks (eight configurations; 4,800 runs). `NoNeg`, `NoDom`, and `NoBeam` disable the named components; hyphenated names disable all listed components. We report score, generation time, generated labels, and `CapHit`, the fraction stopped by the 60-s timeout or 1,000,000-label no-pruning cap.

Table 3 shows why the combination works. Removing Dominance lowers score by 84.98, multiplies time by 2.35, and labels by 2.40. Disabling the entire Beam layer (`NoBeam`), meaning candidate trimming and bucket beam together, lowers score by 166.72, multiplies time by 4.18 and labels by 5.90, and raises limit incidence from 4.67% to 67.67%. Without Beam and Dominance, 91.67% hit a limit and score falls by 572.57, because productive deep search rarely finishes. `NoBeam` is slightly better at 2h but 8.71× slower; every 6h and 8h task times out (Supplementary Table S4).

The Beam layer and Dominance are therefore complementary scalability controls. Negative down-ranking is objective-specific: `NoNeg` ties Full on 594/600 tasks and improves six by only 0.03 points on average, so no generic gain is established here (Supplementary Table S4, Panel B).

5.4 Experiment 3: Balance and Hyperparameter Sensitivity

Balance and rationale. For a method with score s and generation wall time t on a base query, define $g_s = (s_{\max} - s)/(s_{\max} - s_{\min})$ and $g_t = (t - t_{\min})/(t_{\max} - t_{\min})$, where each extremum is taken across methods for that query (a zero range gives zero loss). ReRAPTOR has the smallest mean ideal distance $d = \sqrt{g_s^2 + g_t^2}$ (0.1315, versus 0.7413 for NSGA-II) and is both higher-scoring and faster in 92/120 queries. We test whether $W = 50$, the candidate cap, or label-ranking coefficients drive the result. The two ideal distances and the 92/120 count are reported in Supplementary Table S5.

The one-at-a-time (OAT) panel uses the same 600 tasks per configuration. It tests beam widths of 10, 25, 50, 100, and 200; short/long-budget caps of 15/10, 30/20, and 60/40, or no cap; reward coefficients of 5,000, 10,000, and 20,000; SPH coefficients of 0, 2.5, 5, and 10; effective-time coefficients of 0, 0.05, 0.1, and 0.2; and Negative-penalty scales of 0.5, 1, and 2. The inactive budget-overrun coefficient is not varied because hard feasibility makes its term zero. The candidate-priority weights in Eq. 8 remain fixed and are outside this panel.

OAT differences use exact matched-task pairs and 20,000 crossed block-bootstrap replications over reward seeds and origin–departure blocks. A confirmatory 3×3 panel crosses $W \in \{25, 50, 100\}$ with caps 15/10, 30/20, and 60/40 on a 72-task subset. Runs are sequential on the same workstation; paired score and resources are primary. The OAT panel has 18 configurations (10,800 runs); the interaction panel reuses five OAT center cells and adds four corners (288 runs), totaling 22 configurations and 11,088 verified runs.

Results and interpretation. Figure 4 and Supplementary Table S5 show width stability. $W = 25, 100, 200$ reproduce the default score 872.438 on all 600 tasks; $W = 10$ differs on eight and changes the mean by -0.313 (95% CI $[-0.827, 0]$). Limit incidence rises from 0.00% at $W = 10$ to 2.00%, 3.50%, 6.17%, and 7.00% at $W = 25, 50, 100, 200$, respectively. Thus, 50 is a conservative middle default, not a unique optimum; $W = 25$ is

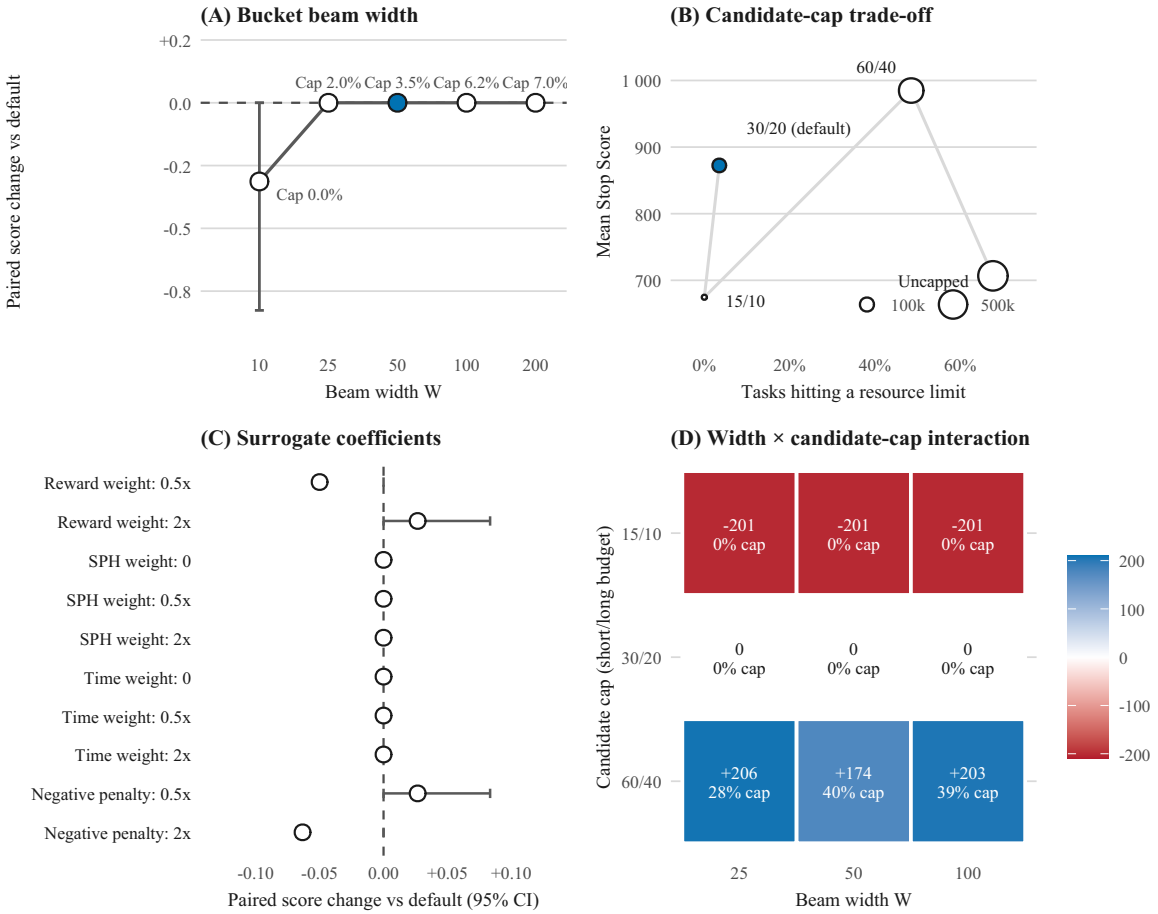


Fig. 4 Canonical hyperparameter sensitivity. (A) Paired score change and 95% crossed block-bootstrap interval across bucket beam widths; labels give resource-cap incidence. (B) Candidate-cap quality–resource trade-off, with point area proportional to generated labels. (C) Paired coefficient effects. (D) Confirmatory width–candidate-cap interaction; cells report mean score change from the $W = 50$, cap-30/20 subset default and cap incidence.

a lower-resource alternative. The taskwise win/tie/loss counts are shown in Supplementary Table S5, Panel B.

Candidate trimming controls the main trade-off. Tight caps 15/10 reduce mean score to 674.56 (paired −197.88, 95% CI [−232.49, −164.07]) but eliminate limit hits. Caps 60/40 raise it to 984.87 (+112.43, 95% CI [+62.18, +171.16]) with 362,108 mean labels and 48.50% limit incidence. With no cap, score falls to 706.69 and 67.67% exhaust a limit. The default 30/20 lies between quality and completion. In the interaction subset, widths remain equivalent at caps 15/10 and 30/20; cap 60/40 adds 174.04–206.33 points but incurs 27.78%–40.28% limit incidence.

Surrogate coefficients have little effect. Halving or doubling the reward weight changes the mean by −0.050 or +0.027; all SPH and time variants reproduce the default; and penalty scales 0.5 and 2 change it by +0.027 and −0.063. This matches the intended reward-first hierarchy. The conclusion is robust over the tested widths and coefficients; the candidate cap sets the material quality–resource boundary.

5.5 Experiment 4: Route Case Study

Three cases selected by fixed rules from the fixed 120-query results explain, rather than extend, the statistics: Case A is the largest lead where ReRAPTOR is faster; B is its smallest score deficit when ranked first by ideal distance; and C is its largest deficit among faster positive-score queries with a ReRAPTOR/NSGA-II ratio of at least 0.8.

Table 4 links results to route behavior. Case A gains mainly by scoring 21 stops rather than 11. In Case B, ReRAPTOR keeps 99.73% of Pure RAPTOR's score while running 94.30× faster. Greedy is faster still (0.85 ms) but scores only 263, so ReRAPTOR provides the intermediate quality–time balance. In Case C, NSGA-II visits one fewer scored stop but earns 84.00 points

Table 4 Route cases. "Stops" counts unique scored non-origin stops; generation time uses the solver timing boundary defined in Section 5.1.

Case	Query	ReRAPTOR			Comparator			Explanation
		Score	Gen. time	Stops	Method	Score	Gen. time	
A	Fukudaimae, 07:00, 8h	1455	13.83 s	21	NSGA-II	684	31.75 s	broader coverage
B	Kashii, 10:00, 2h	375	115.79 ms	6	Pure RAPTOR	376	10.92 s	near both ideals
C	Ohori-koen, 14:00, 8h	1479	4.11 s	21	NSGA-II	1680	50.92 s	lower reward density

per stop versus 70.43; trimming can discard a partial route that later reaches a valuable long-budget sequence. Supplementary Table S6 gives every numerical value used in these comparisons; Supplementary Fig. S1 gives the selection rules and route maps.

6 Discussion

6.1 What the Experiments Establish

The evidence supports a quality–resource claim, not universal route superiority. Experiment 1 shows the aggregate gain and associates it with more distinct scored stops and reliable feasible-route generation, not higher reward per stop. Successful-run comparisons are inconclusive, and several long-budget origins favor NSGA-II.

Experiment 2 supplies the mechanism: the Beam layer bounds outgoing edges and retained labels, while Dominance removes locally weaker labels. Without them, resource limits interrupt deeper search, explaining why their combination can raise completed-route score while reducing time. `NoNeg` is effectively tied with Full, so the optional penalty has no generic benefit here. Experiment 3 shows that this account is stable across tested widths and label-ranking coefficients, while the candidate cap sets the quality–completion boundary. The sensitivity claim is limited to the tested panel: the edge-priority weights in Eq. 8, the 120-min schedule lookahead, the depth rule, the transfer buffer, and the resource thresholds remain fixed.

6.2 Generalisability and Limitations

The seeded rewards create a controlled search benchmark, not an estimate of tourist satisfaction. An application can replace them with attraction ratings, preferences, or event value without changing the timetable backbone. However, independent stationary rewards omit spatial correlation, opening hours, time-dependent value, heterogeneous users, crowding, diversity, and stop interactions. The results concern reproducible value fields, not realized visitor utility.

Two categories define scope. Method-level properties do not rely on Fukuoka: labels retain reward-relevant history, candidate trimming and bucket beam limit different stages, Dominance compares reward and time within a bucket, and the reward module is replaceable. These are component functions, not network-independent effect sizes; Dominance may still discard a history that later becomes valuable. Empirical effect sizes, difficult origins and budgets, limit rates, and useful width–cap settings may depend on Fukuoka. Other service frequencies, transfer patterns, topologies, and observed utilities require tests before any universal performance claim.

7 Conclusion

ReRAPTOR places a pluggable value model inside timetable-feasible expansion, retains route history, and controls the richer search through the Beam layer and heuristic Dominance. On the Fukuoka benchmark, its mean Stop Score is 872.438, 8.710% above NSGA-II, while NSGA-II requires 6.89× as much generation time. Ablation identifies the Beam layer and Dominance as the main scalability controls; sensitivity identifies the candidate cap as the main quality–resource parameter. Successful-run, geographic, and route-case analyses bound the conclusion: ReRAPTOR offers a strong overall balance here, but neither exact optimization nor universal superiority.

Data and Code Availability

The analysis code, canonical derived result files, figure inputs, and reproduction instructions are publicly available at `https://github.com/Akihito-zhou/reraptor_paper`. The evaluated timetable snapshot is identified above by service date and file name, and by a SHA-256 checksum in the repository's reproducibility record. The snapshot was obtained through the Ekispert API and is not redistributed in the public companion. Exact raw-input reproduction requires licensed provider access and the authors' input-edge inventory. The released canonical outputs and verifier support every displayed numerical claim without that proprietary file.

Acknowledgment

This study used transit schedule data for Fukuoka provided through the Ekispert application programming interface (API; Powered by Ekispert API).

Supplementary Materials

Supplementary Tables S1–S7 and Supplementary Fig. S1 in the accompanying `supplementary_materials.pdf` provide numerical traceability for the paired score interpretation, robustness and underperformance analyses, budget-level ablation, sensitivity analysis, balance metrics, route cases, and baseline configurations. Repository paths for every source are printed below the corresponding supplementary table.

References

1. Abuaisha, A., Wallace, M., Harabor, D., Shen, B.: Efficient and exact public transport routing via a transfer connection database. In: Proceedings of the International Symposium on Combinatorial Search, vol. 17, pp. 2–10. Association for the Advancement of Artificial Intelligence (AAAI) (2024). DOI 10.1609/socs.v17i1.31536
2. Agarwal, P., Rambha, T.: Scalable algorithms for bicriterion trip-based transit routing. IEEE Transactions on Intelligent Transportation Systems **25**(10), 14313–14327 (2024). DOI 10.1109/tits.2024.3391343
3. Deb, K., Pratap, A., Agarwal, S., Meyarivan, T.: A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Transactions on Evolutionary Computation **6**(2), 182–197 (2002). DOI 10.1109/4235.996017
4. Delling, D., Dibbelt, J., Pajor, T.: Fast and exact public transit routing with restricted pareto sets. In: 2019 Proceedings of the Twenty-First Workshop on Algorithm Engineering and Experiments (ALENEX), pp. 54–65. Society for Industrial and Applied Mathematics (2019). DOI 10.1137/1.9781611975499.5
5. Delling, D., Dibbelt, J., Pajor, T., Zündorf, T.: Faster transit routing by hyper partitioning. In: 17th Workshop on Algorithmic Approaches for Transportation Modelling, Optimization, and Systems (ATMOS 2017), *Open Access Series in Informatics (OASIcs)*, vol. 59, pp. 8:1–8:14. Schloss Dagstuhl – Leibniz-Zentrum für Informatik (2017). DOI 10.4230/OASIcs.ATMOS.2017.8
6. Delling, D., Pajor, T., Werneck, R.F.: Round-based public transit routing. Transportation Science **49**(3), 591–604 (2015). DOI 10.1287/trsc.2014.0534
7. Demeyer, S., Goedgebeur, J., Audenaert, P., Pickavet, M., Demeester, P.: Speeding up martins' algorithm for multiple objective shortest path problems. 4OR **11**(4), 323–348 (2013). DOI 10.1007/s10288-013-0232-5
8. Euler, R., Lindner, N., Borndörfer, R.: Price optimal routing in public transportation. EURO Journal on Transportation and Logistics **13**, 100128 (2024). DOI 10.1016/j.ejtl.2024.100128
9. Gavalas, D., Konstantopoulos, C., Mastakas, K., Pantziou, G., Vathis, N.: Heuristics for the time dependent team orienteering problem: Application to tourist route planning. Computers & Operations Research **62**, 36–50 (2015). DOI 10.1016/j.cor.2015.03.016
10. Godfrid, J., Radnic, P., Vaisman, A., Zimányi, E.: Analyzing public transport in the city of buenos aires with mobilitydb. Public Transport **14**(2), 287–321 (2022). DOI 10.1007/s12469-022-00290-8
11. Gunawan, A., Lau, H.C., Vansteenwegen, P.: Orienteering problem: A survey of recent variants, solution approaches and applications. European Journal of Operational Research **255**(2), 315–332 (2016). DOI 10.1016/j.ejor.2016.04.059
12. He, P., Jiang, G., Lam, S.K., Sun, Y., Ning, F.: Exploring public transport transfer opportunities for pareto search of multicriteria journeys. IEEE Transactions on Intelligent Transportation Systems **23**(12), 22895–22908 (2022). DOI 10.1109/tits.2022.3194523
13. Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P.: Optimization by simulated annealing. Science **220**(4598), 671–680 (1983). DOI 10.1126/science.220.4598.671
14. Kujala, R., Weckström, C., Darst, R.K., Mladenović, M.N., Saramäki, J.: A collection of public transport network data sets for 25 cities. Scientific Data **5**(1), 180089 (2018). DOI 10.1038/sdata.2018.89
15. Lourenço, H.R., Martin, O.C., Stützle, T.: Iterated local search. In: F. Glover, G.A. Kochenberger (eds.) Handbook of Metaheuristics, *International Series in Operations Research & Management Science*, vol. 57, pp. 320–353. Kluwer Academic Publishers (2003). DOI 10.1007/0-306-48056-5_11
16. Mladenović, N., Hansen, P.: Variable neighborhood search. Computers & Operations Research **24**(11), 1097–1100 (1997). DOI 10.1016/S0305-0548(97)00031-2
17. Pereira, R.H.M., Andrade, P.R., Vieira, J.P.B.: Exploring the time geography of public transport networks with the gtfs2gps package. Journal of Geographical Systems **25**(3), 453–466 (2023). DOI 10.1007/s10109-022-00400-x
18. Persia, F., Pilato, G., Ge, M., Bolzoni, P., D'Auria, D., Helmer, S.: Improving orienteering-based tourist trip planning with social sensing. Future Generation Computer Systems **110**, 931–945 (2020). DOI 10.1016/j.future.2019.10.028
19. Phan, D.M., Viennot, L.: Fast public transit routing with unrestricted walking through hub labeling. In: Analysis of Experimental Algorithms, pp. 237–247. Springer International Publishing (2019). DOI 10.1007/978-3-030-34029-2_16
20. Pilato, G., Persia, F., Ge, M., Chondrogiannis, T., D'Auria, D.: A modular social sensing system for personalized orienteering in the covid-19 era. ACM Transactions on Management Information Systems **14**(4), 1–26 (2023). DOI 10.1145/3615359
21. Ruiz-Meza, J., Montoya-Torres, J.R.: A systematic literature review for the tourist trip design problem: Extensions, solution techniques and future research lines. Operations Research Perspectives **9**, 100228 (2022). DOI 10.1016/j.orp.2022.100228
22. Souffriau, W., Vansteenwegen, P., Vertommen, J., Vanden Berghe, G., Van Oudheusden, D.: A personalized tourist trip design algorithm for mobile tourist guides. Applied Artificial Intelligence **22**(10), 964–985 (2008). DOI 10.1080/08839510802379626
23. Weng, D., Chen, R., Zhang, J., Bao, J., Zheng, Y., Wu, Y.: Pareto-optimal transit route planning with multi-objective monte-carlo tree search. IEEE Transactions on Intelligent Transportation Systems **22**(2), 1185–1195 (2021). DOI 10.1109/tits.2020.2964012
24. Wessel, N., Allen, J., Farber, S.: Constructing a routable retrospective transit timetable from a real-time vehicle location feed and gtfs. Journal of Transport Geography **62**, 92–97 (2017). DOI 10.1016/j.jtrangeo.2017.04.012
25. Wong, J.: Leveraging the general transit feed specification for efficient transit analysis. Transportation Research Record: Journal of the Transportation Research Board **2338**(1), 11–19 (2013). DOI 10.3141/2338-02
26. Xia, D., Ma, J., Sharif Azadeh, S.: Integrated timetabling and vehicle scheduling of an intermodal urban transit network: A distributionally robust optimization approach. Transportation Research Part C: Emerging Technologies **162**, 104610 (2024). DOI 10.1016/j.trc.2024.104610
27. Zhang, Y., Cats, O., Azadeh, S.S.: Dynamic preference-based multi-modal trip planning of public transport and shared mobility. Transportation Research Part E: Logistics and Transportation Review **202**, 104286 (2025). DOI 10.1016/j.tre.2025.104286
28. Zhou, M., Wang, Y.: Optimal routing and request selection for multiple service routes in a demand-adaptive transit system. Frontiers of Engineering Management **12**(4), 983–1004 (2025). DOI 10.1007/s42524-025-4174-3